

Knowledge Base: Embeddings, pgvector, dan RAG

Dokumen contoh untuk validasi pipeline upload + chunking + embedding di Personal Chat AI by Aulia (Roadmap AI Engineer, Minggu 4).

Disusun: Mei 2026 · Voyage AI voyage-3-lite · pgvector 512-dim

1. Apa Itu Embedding?

Embedding adalah representasi numerik (vektor berdimensi tinggi) dari sebuah potongan teks, gambar, atau bentuk data lain. Tujuannya membuat **kesamaan semantik** jadi terukur: dua teks yang artinya mirip akan punya vektor yang berdekatan di ruang vektor, walaupun kata-katanya beda. Misalnya, kalimat “saya suka makan nasi goreng” dan “nasi goreng adalah makanan favorit saya” akan menghasilkan vektor yang sangat mirip, karena memang menyampaikan informasi yang sama walau kata pembuka dan struktur kalimatnya beda. Pencarian biasa (BM25, exact match) akan kesulitan menyamakan keduanya tanpa fitur tambahan.

Model embedding dilatih dengan strategi kontrastif: pasangan teks yang secara logis berhubungan didorong saling dekat, sementara pasangan yang tidak berhubungan didorong saling jauh. Hasil pelatihan tersebut menghasilkan model yang bisa memetakan teks ke titik di ruang vektor — biasanya 256, 512, 768, 1024, atau 1536 dimensi tergantung modelnya. Voyage AI voyage-3-lite yang dipakai di proyek ini menghasilkan vektor 512 dimensi, cukup ringkas tapi cukup ekspresif untuk retrieval multi-bahasa termasuk bahasa Indonesia.

Karena teks panjang sulit di-embed secara akurat (model punya context limit, dan satu vektor harus mewakili semua isi teks), biasanya teks dipotong dulu jadi **chunks** berukuran ratusan kata sebelum di-embed satu per satu. Cara memotong ini punya pengaruh besar terhadap kualitas pencarian: chunk yang terlalu kecil kehilangan konteks, chunk yang terlalu besar membuat vektor terlalu kabur. Strategi *heading-aware* mengikuti struktur dokumen, yang umumnya menghasilkan chunk yang koheren secara topik.

Setelah teks di-embed, vektor disimpan di database vektor (kita pakai pgvector, extension Postgres). Untuk pencarian, query juga di-embed dengan model yang sama (atau dengan input_type berbeda untuk model yang mendukung asymmetric retrieval), lalu kita cari top-k vektor terdekat di database. Hasil top-k itu biasanya jadi konteks yang disuntikkan ke prompt LLM — itulah inti dari Retrieval-Augmented Generation (RAG).

2. Cara Kerja pgvector

pgvector adalah extension Postgres yang menambahkan tipe data vector(N) dan operator pencarian jarak. Setelah CREATE EXTENSION vector; di-jalankan, kita bisa bikin kolom seperti embedding vector(512) lalu insert nilai dalam bentuk literal '[0.1, 0.2, ...]::vector'. Postgres memperlakukan vektor seperti tipe asli lain, dengan tambahan operator khusus

untuk perhitungan jarak.

Tiga operator utama yang paling sering dipakai adalah: `<->` untuk Euclidean distance, `<#>` untuk negative inner product, dan `<=>` untuk cosine distance. Cosine distance adalah pilihan default untuk teks karena dia mengukur sudut antar vektor, bukan magnitude — artinya dia robust terhadap panjang teks. $\text{Similarity} = 1 - \text{cosine_distance}$, berkisar dari -1 (kebalikan) sampai 1 (identik); untuk teks alami biasanya ada di range 0..1.

Tanpa index, query `ORDER BY embedding <=> $1 LIMIT 5` akan full-scan tabel — OK untuk dataset kecil tapi tidak skala. Untuk skala, pgvector menyediakan dua tipe index: **IVFFlat** dan **HNSW**. IVFFlat membutuhkan training (perlu data dulu sebelum bikin index), sedangkan HNSW lebih sederhana dan biasanya lebih cepat untuk recall tinggi, dengan trade-off build time yang lebih lambat. Untuk pemakaian umum personal, HNSW dengan parameter default ($m=16$, $ef_construction=64$) sudah lebih dari cukup.

Saat query, parameter `hnsw.ef_search` menentukan berapa banyak kandidat yang dievaluasi pada saat pencarian — nilai yang lebih tinggi memberi recall lebih baik tapi lebih lambat. Default 40 biasanya OK; kalau perlu recall tinggi naikkan ke 100-200. Kalau ada filter (misal `WHERE user_id = $1`) yang sangat selektif, pertimbangkan partial index atau combine dengan strategi lain seperti pre-filter SQL sebelum vector similarity sort.

3. Strategi Chunking

Chunking adalah proses memotong dokumen jadi potongan-potongan kecil yang bisa di-embed sebagai unit independen. Tujuannya bukan sekadar mematuhi context limit model embedding, tetapi juga memaksimalkan *signal-to-noise ratio* per chunk: ideally, satu chunk berisi satu topik yang koheren, tanpa filler yang tidak relevan. Kalau chunk berisi banyak topik, vektornya jadi rata-rata dari semua topik tersebut — kabur dan kurang berguna untuk retrieval spesifik.

3.1 Heading-aware split

Strategi paling natural untuk dokumen markdown adalah memotong di setiap heading (#, ##, ###). Heading menandai perubahan topik secara eksplisit, dan section di bawah heading biasanya membahas satu sub-topik. Implementasinya: iterasi baris demi baris, kalau ketemu baris yang dimulai dengan # diikuti spasi, anggap itu heading dan flush buffer ke chunks list. Heading-nya tetap di-include di awal chunk supaya konteks topik ikut ter-embed.

3.2 Fallback fixed-size split

Tidak semua dokumen punya heading (plain text, transcript, dump dari email, dll). Dan kalau pun ada heading, section di bawahnya bisa terlalu panjang untuk satu chunk. Untuk kasus seperti ini, kita perlu fallback splitter yang memotong by karakter. Aturan praktis: target ~1500 karakter per chunk dengan 100 karakter overlap antar chunk. Overlap membantu mempertahankan konteks kalau informasi penting jatuh pas di batas chunk.

Saat memotong by karakter, usahakan potong di whitespace atau tanda baca, bukan di tengah kata. Algoritmenya sederhana: setelah menentukan target posisi end, scan mundur sampai ketemu space, newline, period, atau koma. Kalau gagal ketemu dalam jarak masuk akal (~max/2 karakter), ya potong di posisi mentah saja — lebih baik chunk sedikit tidak rapi daripada chunk satu kata.

3.3 Merge small chunks

Section yang sangat pendek (intro 1-2 kalimat) menghasilkan chunk yang terlalu kecil, dan vektor-nya mungkin tidak punya cukup informasi untuk retrieval yang berguna. Setelah split, lakukan pass untuk merge chunk-chunk yang panjangnya di bawah threshold (~200 karakter) ke chunk sebelumnya — tapi hanya kalau headingnya sama, supaya batas topik tetap terjaga.

4. Pattern RAG

Retrieval-Augmented Generation (RAG) adalah teknik di mana LLM diberi konteks tambahan yang di-retrieve dari knowledge base eksternal saat query, alih-alih hanya bergantung pada training data internal model. Manfaat utamanya: jawaban bisa di-grounding ke sumber tertulis (citation-able), knowledge base bisa di-update tanpa fine-tuning, dan model bisa menjawab tentang dokumen private yang tidak pernah ada di training data.

Pipeline RAG paling sederhana cuma 4 langkah: (1) chunking & embedding knowledge base saat ingest, (2) embedding query saat user kirim pesan, (3) similarity search top-k chunks, (4) inject top-k chunks ke system prompt atau user message lalu kirim ke LLM. Variasinya banyak: re-ranking dengan cross-encoder, hybrid search (BM25 + vector), query rewriting, hierarchical retrieval, parent-document retrieval, dan masih banyak lagi.

Salah satu trade-off penting di RAG adalah *retrieval precision vs recall*. Kalau kita ambil top-3 chunks, kemungkinan dapet chunks yang super relevan tinggi (precision), tapi mungkin ada info penting yang ke-skip (recall rendah). Kalau ambil top-20, recall naik tapi precision turun — context window LLM kepenuhan dengan chunk yang setengah relevan. Best practice: pakai re-ranker yang lebih akurat (tapi lebih lambat) di tahap kedua, ambil top-10 dari vector search, lalu pilih top-3 dari re-ranker.

Untuk citation, simpan metadata chunk (document_id, position, heading) dan tampilkan di response. Pattern yang umum: LLM diminta cite source pakai format [1], [2] di akhir kalimat. Di FE, render marker tersebut sebagai link clickable yang membuka chunk asli. Ini bikin user bisa verifikasi jawaban dan klarifikasi konteks. Tahap ini direncanakan untuk Minggu 5 di roadmap.

5. Tips Implementasi

Asymmetric embedding. Banyak model embedding modern (termasuk Voyage) mendukung parameter input_type yang berbeda untuk dokumen vs query (“document” vs “query”). Set ini dengan benar saat ingest dan saat search — ini bukan optional, perbedaannya bisa signifikan untuk kualitas retrieval.

Batch embeddings. Voyage AI menerima sampai 128 dokumen per request. Selalu batch saat ingest dokumen besar — satu request untuk 128 chunks jauh lebih efisien dari 128 request masing-masing 1 chunk. Untuk query (single text), batch tidak diperlukan.

Embedding model migration. Kalau suatu saat mau ganti model embedding (misal dari voyage-3-lite ke voyage-3, atau ke OpenAI text-embedding-3-large), perlu re-embed semua chunks dan ubah dimensi kolom vector. Simpan model yang dipakai di field embedding_model per dokumen supaya mudah tracking dokumen mana yang sudah migrate.

HNSW index timing. Bikin index *setelah* bulk insert chunks yang banyak, bukan sebelumnya. HNSW build cepat untuk dataset kecil-menengah, tapi update incremental per row lebih lambat. Untuk personal use yang mostly 1-2 dokumen per hari, perbedaannya tidak signifikan.